

Week -8 PRATICAL

WEEK:8

Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

Develop a program that demonstrates Copy-on-Write behavior after a fork(). Analyze memory usage before and after modifying data in the child process.

Dynamic Memory + Valgrind

nano dynamic_memory.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main()
{
    printf("==== DYNAMIC MEMORY DEMONSTRATION =====\n\n");

    /* malloc() */
    int *arr = malloc(5 * sizeof(int));

    if (arr == NULL)
    {
        printf("malloc failed\n");
        return 1;
    }

    printf("malloc(): Memory allocated for 5 integers.\n");

    for (int i = 0; i < 5; i++)
    {
        arr[i] = (i + 1) * 10;
    }

    printf("Values using malloc(): ");

    for (int i = 0; i < 5; i++)
    {
        printf("%d ", arr[i]);
    }

    printf("\n\n");

    /* calloc() */
    int *zeros = calloc(5, sizeof(int));

    if (zeros == NULL)
    {
        printf("calloc failed\n");
        free(arr);
    }
}
```

```

    return 1;
}

printf("calloc(): Memory allocated for 5 integers.\n");
printf("Initial calloc values: ");

for (int i = 0; i < 5; i++)
{
    printf("%d ", zeros[i]);
}

printf("\n\n");

/* realloc() */
arr = realloc(arr, 10 * sizeof(int));

if (arr == NULL)
{
    printf("realloc failed\n");
    free(zeros);
    return 1;
}

printf("realloc(): Memory resized from 5 to 10 integers.\n");

for (int i = 5; i < 10; i++)
{
    arr[i] = (i + 1) * 10;
}

printf("Values after realloc(): ");

for (int i = 0; i < 10; i++)
{
    printf("%d ", arr[i]);
}

printf("\n\n");

/* free() */
free(arr);
free(zeros);

printf("free(): Allocated memory released.\n");

printf("\n==== PROGRAM COMPLETED =====\n");

return 0;
}

```

```

bharath@DESKTOP-6D34C2R:~/PRATICAL/week8$ nano dynamic_memory.c
bharath@DESKTOP-6D34C2R:~/PRATICAL/week8$ gcc -Wall -Wextra dynamic_memory.c -o dynamic_memory
bharath@DESKTOP-6D34C2R:~/PRATICAL/week8$ ./dynamic_memory
===== DYNAMIC MEMORY DEMONSTRATION =====

malloc(): Memory allocated for 5 integers.
Values using malloc(): 10 20 30 40 50

calloc(): Memory allocated for 5 integers.
Initial calloc values: 0 0 0 0 0

realloc(): Memory resized from 5 to 10 integers.
Values after realloc(): 10 20 30 40 50 60 70 80 90 100

free(): Allocated memory released.

===== PROGRAM COMPLETED =====
bharath@DESKTOP-6D34C2R:~/PRATICAL/week8$ valgrind --leak-check=full ./dynamic_memory
Command 'valgrind' not found, but can be installed with:
sudo snap install valgrind # version 3.26.0, or
sudo apt install valgrind # version 1:3.26.0-0ubuntu1
See 'snap info valgrind' for additional versions.

```

nano leak.c

```

#include <stdio.h>
#include <stdlib.h>

```

```

int main()
{
    int *ptr = malloc(100 * sizeof(int));

    if (ptr == NULL)
    {
        return 1;
    }

    for (int i = 0; i < 100; i++)
    {
        ptr[i] = i;
    }

    printf("Memory allocated but not freed.\n");

    return 0;
}

```

```

bharath@DESKTOP-6D34C2R:~/PRATICAL/week8$ nano leak.c
bharath@DESKTOP-6D34C2R:~/PRATICAL/week8$ gcc -Wall -Wextra leak.c -o leak
bharath@DESKTOP-6D34C2R:~/PRATICAL/week8$ ./leak
Memory allocated but not freed.
bharath@DESKTOP-6D34C2R:~/PRATICAL/week8$ valgrind --leak-check=full ./leak
Command 'valgrind' not found, but can be installed with:
sudo snap install valgrind # version 3.26.0, or
sudo apt install valgrind # version 1:3.26.0-0ubuntu1
See 'snap info valgrind' for additional versions.

```

Copy-on-Write using fork()

```
nano cow.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define SIZE 1000000

int global_array[SIZE];

int main()
{
    for (int i = 0; i < SIZE; i++)
    {
        global_array[i] = i;
    }

    printf("Parent PID: %d\n", getpid());
    printf("Address of global_array: %p\n",
        (void *)global_array);

    printf("Before fork(): global_array[0] = %d\n",
        global_array[0]);

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid == 0)
    {
        printf("\nChild process\n");
        printf("Child PID: %d\n", getpid());

        printf("Before modification:\n");
        printf("global_array[0] = %d\n",
            global_array[0]);

        printf("Address in child: %p\n",
            (void *)global_array);

        printf("\nChild modifying global_array[0]...\n");

        global_array[0] = 999;

        printf("After modification:\n");
        printf("global_array[0] = %d\n",
            global_array[0]);
    }
}
```

```

    printf("Address in child after modification: %p\n",
           (void *)global_array);

    exit(0);
}
else
{
    wait(NULL);

    printf("\nParent process\n");

    printf("Parent global_array[0] = %d\n",
           global_array[0]);

    printf("Address in parent: %p\n",
           (void *)global_array);
}

return 0;
}

```

See 'snap info valgrind' for additional versions.

```

bharath@DESKTOP-6D34C2R:~/PRATICAL/week8$ nano cow.c
bharath@DESKTOP-6D34C2R:~/PRATICAL/week8$ gcc -Wall -Wextra cow.c -o cow
bharath@DESKTOP-6D34C2R:~/PRATICAL/week8$ ./cow
Parent PID: 617
Address of global_array: 0x6483ecab8040
Before fork(): global_array[0] = 0

Child process
Child PID: 618
Before modification:
global_array[0] = 0
Address in child: 0x6483ecab8040

Child modifying global_array[0]...
After modification:
global_array[0] = 999
Address in child after modification: 0x6483ecab8040

Parent process
Parent global_array[0] = 0
Address in parent: 0x6483ecab8040
bharath@DESKTOP-6D34C2R:~/PRATICAL/week8$ |

```